CtrlPay — Technical Documentation

2026 Shenzhen FinTechathon · International Track · Topic A (Personal Finance / Defensive AI)
Submission Checklist Item 01 — Technical Documentation

CtrlPay is a defensive personal-finance agent for Malaysian university students. It protects the user from impulse spending and forgotten ("zombie") subscriptions. Every monetary figure, probability and percentage is produced by a deterministic TypeScript engine; the large language model (LLM) extracts intent and identifiers only, and the human always makes the final decision.

This document covers the three areas required by the checklist: **System Architecture**, **Core Algorithm Description**, and **Security Design**.

---

1. System Architecture

1.1 Two-Brain Server-Side Authority

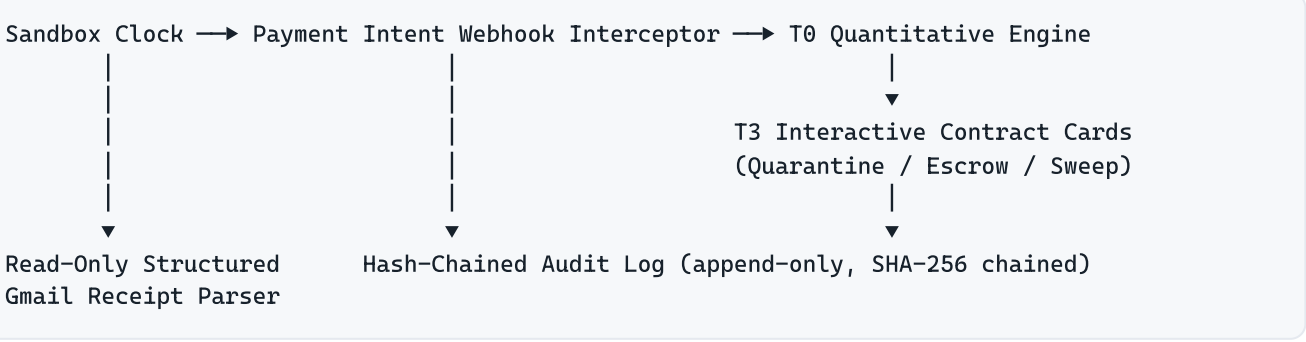
CtrlPay deliberately separates *language* from *arithmetic*:

Brain	Where it lives	Responsibility	What it must never do
Language brain	`app/api/chat/route.ts` + `lib/groq-client.ts` (Groq Chat Completions, GPT-OSS 120B; AI SDK v4 stream bridge)	Understand the user, extract intent + identifiers, call tools, write prose	Compute or invent any money figure, see the signing secret, or hold autonomous credentials
Arithmetic brain	`lib/quant-engine.ts` (pure TypeScript)	Compute every RM figure, probability, ratio and classification deterministically	Call the network or the LLM

The LLM can only reference a subscription by the `subscriptionId` enum (`lib/subscription-registry.ts`); it can never supply a URL. The server computes a proposal, signs it with HMAC-SHA256 (`lib/signing.ts`), and only then does the LLM narrate the already-computed numbers. A post-generation verifier (`verifyNumbers`) rejects any reply that contains an `RM` figure absent from the engine output for that turn.

1.2 Proactive, Event-Driven Sandbox

CtrlPay is *proactive*: instead of waiting to be asked, it reacts to events and intervenes before money leaves the account.



- **Sandbox Clock** (`lib/store.ts` , `lib/utils.ts`): a simulated wall clock starting at `2026-03-12T09:00:00Z` . The simulation toolbar advances it by **+24 hours**, **+7 days**, or **Jump to Month-End**. On each tick, `resolveClock` fires scheduled effects (12:00 PM daily spending summaries, billing-date crossings that settle pending sweeps, month-end roast metrics).
- **Payment Intent Webhook Interceptor** (`simulatePaymentIntent`): models a bank/payment webhook. It runs the T0 What-If simulation and, when risk is HIGH/CRITICAL or a user limit is breached, emits an intercept event that opens the T3 Contract Card.
- **Read-Only Structured Gmail Receipt Parser** (mock, `lib/bookkeeping.ts`): an opt-in, revocable, read-only path that yields *structured fields only* (merchant, amount, category) and never raw email bodies.
- **T0 Quantitative Engine** (`lib/quant-engine.ts`): computes Cash-Flow-at-Risk, the dynamic daily allowance, amortized subscription burden and the intervention gate.
- **T3 Three-Option Interactive Contract Cards** (`components/contract-cards.tsx`): every intervention is presented as an explicit **Approve / Modify / Decline** decision (plus a dismiss fallback), each carrying a signed proposal.
- **Hash-Chained Audit Log** (`lib/audit-logger.ts`): append-only, tamper-evident, one chain per tenant.

1.3 End-to-End Module & Route Architecture

Libraries (`lib/`)

File	Role
`types.ts`	Zod schemas + inferred types for tools, proposals, decisions, mandates and limits
`domain.ts`	Domain interfaces (FixedBill, Subscription, WhatIfInput/Result, EscrowFloorCheck, …)
`quant-engine.ts`	**Deterministic arithmetic brain** — CFaR Monte Carlo, allowance, escrow floor, amortization, reward, routing
`prng.ts`	Seeded `mulberry32` PRNG + `randInt` (reproducible simulations)
`utils.ts`	`round2` , RM formatting, ISO date helpers, `remainingDaysInMonth` , `noonsBetween` , `endOfMonthMs`
`sha256.ts`	Dependency-free isomorphic SHA-256 (verified against `node:crypto`)
`canonical.ts`	Deterministic JSON canonicalisation for hashing
`signing.ts`	**Server-only** HMAC-SHA256 proposal signing + verification
`ids.ts`	ID generation
`ledger.ts`	`LedgerSnapshot` build/parse, upcoming fixed bills, What-If input assembly
`subscription-registry.ts`	Hardcoded, team-maintained subscription whitelist (prices + HTTPS deep links)
`zombie-radar.ts`	Merchant normalisation, recurring-charge detection, zombie tagging, student-arbitrage detection
`privacy-guard.ts`	Pre-LLM PII redaction + post-LLM RM number verifier

File	Role
`audit-logger.ts`	Hash-chained, per-tenant append-only log; `mergeExternal` for server→client unification
`audit-sync.ts`	Client helper: pull server AGENT entries and merge them into the local chain
`session.ts`	**Server-only** signed-cookie session → `userId` (tenant isolation)
`permissions.ts`	Permission tiers T0–T3, `TOOL_TIERS`, `assertTier`, click-authorization regex
`proposals.ts`	**Server-only** proposal creation + `decideProposal` (integrity, tenant, expiry, floor, mandate)
`server-tools.ts`	Server tool implementations (all numbers from `quant-engine`)
`system-prompt.ts`	Locked system prompt + `buildFallbackReply`
`groq-client.ts`	**Server-only** Groq Chat Completions client, model retirement guard, OpenAI-compatible `tools` conversion, T3 call-context codec
`mock-data.ts`	Deterministic sandbox fixtures (balance, subscriptions, transactions, receipts, usage)
`store.ts`	Zustand client state: clock resolution, decisions, sweeps, quarantine, booking, audit writes
`bookkeeping.ts`	`parseBookingText`, `mockReceiptExtraction` (name-only deterministic fixture)
`tool-resolution.ts`	Contract-card decision builders + dangling-tool-call guard
`api-client.ts`	Browser fetch helpers for proposals, decisions and `mailto:` drafts

API routes (`app/api/`)

Route	Verb	Purpose
`/api/chat`	POST	Groq Chat Completions (GPT-OSS 120B default); T0–T2 tools executed server-side, T3 tools returned as cards and resumed with `addToolResult`; AI SDK v4 data stream; post-LLM number verifier
`/api/ai-analyze`	POST	Advisory AI receipt red-flag check (`ocr_receipt`) and payment-interceptor advice (`payment_advice`); never mutates the ledger
`/api/proposals`	POST	Create + HMAC-sign a proposal (server computes all terms)
`/api/proposals/[id]/decision`	POST	Decide a proposal; verifies integrity, tenant, expiry, T3 gate, escrow floor and AI mandate. Accepts the echoed signed proposal for **stateless** verification
`/api/audit`	GET	Returns server-side AGENT audit entries for the caller's tenant

Components (`components/`)

Component	Role
`agent-chat.tsx`	AI SDK v4 `useChat`, quick chips, roast-tone toggle, contract-card mounting, verifier banner

Component	Role
`contract-cards.tsx`	Quarantine / Escrow / Sweep cards (Approve / Modify / Decline) + guaranteed tool resolution
`payment-interceptor-modal.tsx`	Payment-intent interceptor with the three T3 options and live Groq advisory
`ocr/receipt-ai-flow-modal.tsx`	Receipt OCR → editable review → live AI red-flag check and warning popup
`quarantine-cards.tsx`	Active quarantine list + "Abandon Purchase & Claim Reward"
`vault-panel.tsx`	`defenseVault` + `rewardPool` ledger buckets
`subscription-timeline.tsx`	Upcoming billing timeline
`zombie-radar-panel.tsx`	Zombie tags + student-downgrade sweep entry
`student-downgrade-drawer.tsx`	Student-email/OTP check + downgrade sweep flow
`multimodal-bookkeeping.tsx`	Receipt upload, voice and typed booking (confirm-before-book)
`event-feed.tsx`	Raw `[SANDBOX]` event stream
`audit-drawer.tsx`	Unified hash-chained audit log with chain-verified badge
`simulation-toolbar.tsx`	Clock controls + Master AI Autonomy Switch
`mobile-app-shell.tsx`	**Phase 8 mobile shell** — the captain's e-wallet visual language on CtrlPay data: phone frame, 5-tab bottom nav (Home / Timeline / Radar / Bookkeep / Copilot & Security), balance hero, needs-vs-wants ring, dual limits, AI mandate sliders, 12:00 summary, commitment devices
`cfar-stress-tester.tsx`	Interactive What-If CFaR tester (shortfall, CFaR95, ES95, mid-month NSF dip, stratified bootstrap, pre-auth latency)
`risk-tail-strip.tsx`	Shared fintech telemetry strip reused by the interceptor and the contract cards (ES95, NSF risk, pre-auth ms, present-bias markup, risk-elastic reward)
`ui/*`	Badge, Button, Card, Slider, Switch primitives

Layout modes (`app/page.tsx`)

The single-page judge experience ships three synchronized views over the same store:

Mode	Purpose
App + Judge Studio (default)	The student mobile app on the left, the judge console (copilot chat, simulation toolbar, audit log, telemetry, vault) on the right
Full App Focus	Phone-first demo; judge controls and the audit log slide in as drawers, and the copilot chat is mounted inside the phone's Copilot tab
Expanded Dashboard	Full-width multi-column inspection of every panel and log

The copilot chat is mounted in exactly one place per mode, so the AI SDK stream, tool-call state and the audit chain stay single-sourced. The phone shell is a faithful port of the team's earlier dual-currency e-wallet prototype (`docs/legacy/duoMoney_Plus_v36.html`) — same gradients, 22px cards, budget-bar palette and bottom navigation — but every figure on screen is produced by the deterministic engine from the sandbox ledger, never by the LLM.

---

2. Core Algorithm Descriptions

All algorithms below live in `lib/quant-engine.ts` (or the named module) and are fully deterministic: identical inputs always produce identical outputs.

2.1 Historical Bootstrap Cash-Flow-at-Risk (CFaR) Monte Carlo

Implemented by `runWhatIfSimulation(input: WhatIfInput)`. Let

- `B` = current balance,
- `U` = total **unpaid** fixed bills,
- `P` = proposed expense amount,
- `G` = monthly savings goal,
- `D` = remaining days in the month,
- `H` = historical daily spending vector (length `m`).

We draw **N = 1,000** bootstrap paths using a seeded `mulberry32(seed)` generator. For path *i*:

```
S_i = Σ_{j=1..D} H[ randInt(rng, m) ]        (simulated remaining spend)
E_i = B − U − P − S_i                        (projected month-end balance)
L_i = max(0, G − E_i)                        (shortfall vs. savings goal)
```

Then:

- `projectedMonthEndBalance = round2( (1/N) Σ E_i )`
- `shortfallProbability = round2( 100 · #{E_i < G} / N )` (%)
- `negativeProbability = 100 · #{E_i < 0} / N` (%)
- `CfaR95 = round2( percentile_95( sortedAscending(L_i) ) )` where `percentile(sorted, p) = sorted[ min(len−1, ceil(p·len) − 1) ]`.

Risk classification (`classifyRisk`), with thresholds `low = 10%` , `medium = 30%` , `high = 60%` and `criticalNegativeProbability = 20%` :

Condition	Level
`shortfallProbability ≥ 60` **or** `negativeProbability > 20`	`CRITICAL`
`shortfallProbability ≥ 30`	`HIGH`
`shortfallProbability ≥ 10`	`MEDIUM`
otherwise	`LOW`

Any weekly/monthly user-limit breach raises the level to at least `HIGH` via `maxRisk`. When fewer than `MIN_HISTORY_DAYS = 14` days of history exist, the bootstrap falls back to a flat estimate (the historical mean, or `campusMixedRice × 2 = RM 18/day` when there is no history) and sets `lowHistoryWarning = true`.

The **dynamic daily allowance** is:

```
dynamicDailyAllowance = round2( max(0, (B − U − savingsGoal − proposedExpense) / max(1, D) )
)
```

2.1.1 Phase 8 tail-risk extensions

95% Expected Shortfall (Basel III / FRTB; Artzner et al., 1999). From the ascending shortfall sample, with `N = 1000` paths (1-based order statistics `L_(i)`):

```
es95 = round2( (1/|T|) · Σ_{i ∈ T} L_(i) ),   T = { i : i ≥ ceil(0.95 · N) }
```

i.e. the mean of the worst 5% of month-end shortfalls — the expected loss *given* the loss has breached its 95% quantile. ES is a **coherent** risk measure (monotone, sub-additive, positively homogeneous) whereas a bare quantile is not, so `es95 ≥ cfar95` holds by construction on the same sample; the Phase 8 test suite asserts the coherence relation for every scenario. This is also the CVaR estimator of Rockafellar & Uryasev (2000), whose convex-programming representation is what made tail-mean risk the standard replacing quantiles in Basel III's FRTB internal-models approach: the worst-5% mean is directly minimisable, while a quantile is not.

Intra-month liquidity barrier (Basel III LCR analogue). A month-end-only projection is mathematically blind to mid-month non-sufficient-funds (NSF) events: with no committed inflow a running balance is monotone non-increasing, so `P(min B(t) < 0)` would collapse into `P(month-end < 0)`. The barrier therefore models the **scheduled allowance inflow** (DEMO_CONFIG: RM 800 landing on day 25) and records, for each of the same 1,000 paths:

```
M_i     = min over t of B_i(t)
B_i(t)  = B − P − Σ{bills due ≤ t} − Σ{daily spend ≤ t} + (allowance if t ≥ allowanceDay)

intraMonthLiquidityBreachProb        = round2( 100 · #{ M_i < 0 } / N )      (%)
lowestProjectedBalanceRM             = round2( percentile_5( sortAsc(M_i) ) )
projectedMonthEndBalanceWithIncomeRM = round2( (1/N) · Σ B_i(D) )
```

The month-end CFaR figures (`projectedMonthEndBalance`, `shortfallProbability`, `cfar95`) deliberately do **not** credit future inflows — they remain the conservative solvency view — while the barrier is the timing-aware liquidity view. Both are reported side by side, mirroring how Basel III separates a full capital horizon from the 30-day LCR window. Bills whose `dueDay` already passed, or falls past the remaining horizon, are

excluded from the barrier and stay inside `U`. Without a sandbox clock the engine falls back to a deterministic 30-day window aligned so `remainingDaysInMonth` still matches the bills' `dueDay` values.

Day-of-week stratified bootstrap (Künsch, 1989). Weekend spending is structurally heavier, so the history is partitioned by index (`i % 7 ∈ {5,6}` → weekend pool) and horizon day *t* is classified by its real weekday (from the sandbox clock); each daily draw then comes from the matching pool. A **separate seeded PRNG instance** (`seed ^ 0x9e3779b9`) drives this resample, so the primary `shortfallProbability` / `cfar95` draw sequence — and therefore every pre-Phase-8 output — stays byte-identical. The result is reported as `stratifiedShortfallProbability`.

Pre-authorization latency telemetry. `executionTimeMs` is a genuine `performance.now()` delta around the pure model, taken on a warmed second pass (best-of-2 estimator, excluding one-time JIT warm-up), and it is **memoized with the result**: identical inputs report byte-identical telemetry, which is what the deterministic-engine tests and the evidence generator rely on. The engine target is < 5 ms per check against a hard product budget of 50 ms — the payment-rail pre-authorization window the interceptor sits in.

2.2 Sinking Fund — Annual/Quarterly Fee Amortization

Implemented by `calculateAmortizedSubscriptions(subs, options?: { sandboxClock?: number })`. Recurring fees are smoothed into a monthly and daily burden so they hit the allowance every day rather than all at once:

```
monthlyBurden = amountRM / 12      (ANNUAL)
              = amountRM / 3       (QUARTERLY)
              = amountRM           (MONTHLY)

dailyBurden   = monthlyBurden × 12 / 365
setAsideThisMonth = monthlyBurden
```

The total monthly burden feeds `computeDynamicDailyAllowance` through the `amortizedMonthlyBurden` argument, so enabling amortization in the UI lowers the safe daily allowance in real time. (Example: the RM 180/year gym membership becomes RM 15.00/month, or ~RM 0.49/day.)

Phase 8 maturity liquidity gate (treasury spot-cash reserve). Amortization is an accounting smoothing device, not cash. When a sandbox clock is supplied the function also returns

```
daysUntilBilling      = ceil( (nextBillingDate − sandboxClock) / 86 400 000 )
maturityGateTriggered = billingCycle ≠ MONTHLY  ∧  0 ≤ daysUntilBilling ≤ 7
```

and the Visual Subscription Timeline raises a **Spot Cash Reserve Gate** banner: for example the RM 180/year gym charge is RM 15.00 in the sinking-fund view, yet RM 180.00 of *spot cash* is due inside the settlement window — a classic maturity mismatch the gate makes explicit. Monthly charges never trigger the gate (no maturity transformation), and without a clock both fields are omitted so all pre-Phase-8 call sites and outputs are unchanged.

2.3 Needs vs. Wants Protection Floor

Implemented by `validateEscrowFloor({ lockedCategories, dailyBudgetReduction, dynamicDailyAllowance, durationDays, mandate })`. The floor is enforced at two levels:

1. **Type level.** `NeedsCategorySchema` (`Bills_Utilities`, `Campus_Cafeteria_Basic`, `Medical`, `Transport_Essential`) is disjoint from `WantsCategorySchema` (`Food_Delivery_Grab_Foodpanda`,

`Entertainment`, `Bubble_Tea_Dining`, `Shopping`). `isNeedCategory()` provides a runtime backstop; if any NEEDS category appears, the result is `NEEDS_CATEGORY_LOCKED`.

2. **Runtime level.**

```
byRatio            = 0.40 × dynamicDailyAllowance            (40% maximum reduction)
byFloor            = dynamicDailyAllowance − 15.0            (RM 15/day survival floor)
maxAllowedReduction= round2( max(0, min(byRatio, byFloor)) )
```

- `maxAllowedReduction ≤ 0` → `BELOW_SURVIVAL_FLOOR`.
- `dailyBudgetReduction > maxAllowedReduction` → `REDUCTION_OVER_40_PERCENT` when it exceeds `byRatio`, and/or `BELOW_SURVIVAL_FLOOR` when `dda − reduction < 15`.
- `durationDays > mandate.maxLockDurationDays` **or** `dailyBudgetReduction × durationDays > mandate.maxLockAmountRM` → `EXCEEDS_AI_MANDATE`.

The function also returns the rounded `maxAllowedReduction`, which the modify flow re-signs into the new escrow terms (this fixes a rounding path where the cap itself could be exceeded).

Stone-Geary subsistence floor (Phase 8). The result also exposes the subsistence layer of a Stone-Geary (1954) linear expenditure system explicitly:

```
subsistenceFloorRM           = 15.00        (γ, incompressible NEEDS consumption in RM/day)
stoneGearyMarginalUtilitySafe = (dda − dailyBudgetReduction) ≥ γ
```

Below γ the marginal utility of the remaining allowance is dominated by survival needs, so any escrow that would push the user under it is rejected; above γ the consumer allocates *supernumerary* income across wants at constant marginal budget shares.

2.4 Two-Stage Sweep Settlement & Anti-Farming Quarantine State Machine

Two-stage sweep settlement. A subscription savings sweep does not credit the vault immediately:

```
Stage 1  create  : PendingSweep { monthlySavings, status: PENDING_NEXT_BILLING_VERIFICATION }
Stage 2  settle  : on the next billing-date crossing, the sweep becomes VERIFIED and
                   defenseVault += monthlySavings
                   rewardPool   += monthlySavings
```

`defenseVault` is a labelled, **non-interest-bearing ledger bucket** — no real money moves and no yield is fabricated. The `rewardPool` is funded **only** by VERIFIED sweep savings. Because both buckets move by exactly the verified `monthlySavings`, the invariant *vault growth ≤ verified savings* always holds. Manually-processed contracts (e.g. Anytime Fitness, a gym with no self-service cancellation) resolve to `NOT_VERIFIED` and credit **nothing** (`SWEEP_NOT_VERIFIED`).

Anti-farming quarantine state machine.

```
PROPOSED —(user clicks Approve, valid USER_CLICK_APPROVE_PROPOSAL_* ref)→ LOCKED_24H
LOCKED_24H —(advanceClock +24h)→ READY_TO_DECIDE
READY_TO_DECIDE —("Abandon Purchase & Claim Reward")→ CLAIMED
```

The reward is `reward = round2( max(0, min( itemPrice × 0.05, RM 15.00, rewardPool )) )`, and on claim `rewardPool −= reward`, `defenseVault += reward`. The claim button is disabled until the sandbox

clock has advanced a full 24 hours, so the reward cannot be farmed by locking and immediately releasing. `releaseQuarantine` (proceed with the purchase) awards no reward.

Behavioral layer (Laibson, 1997) — why a 24-hour wait actually works.
`calculateBehavioralMetrics(itemPrice, shortfallProbability, rewardPoolBalance)` returns:

```
presentBiasBeta           = β = 0.70          (quasi-hyperbolic discount factor)
dailyDiscountDelta        = δ = 0.999
presentBiasInflationRatio = round2(1 / β) = 1.43×
baseRewardRM              = calculateQuarantineReward(itemPrice, rewardPool)      (unchanged)
riskElasticRewardRM       = round2( min( itemPrice × 0.05 × (1 + 0.5 ×
shortfallProbability/100),
                                      RM 15.00, rewardPool ) )
```

Under quasi-hyperbolic discounting `U_t = u(c_t) + β·Σ_{k≥1} δ^k·u(c_{t+k})` an *immediate* purchase is perceived `1/β = 1.43×` more valuable than the identical purchase delayed by 24 hours. The Quarantine is designed to cancel exactly that markup: the user either collects a reward for waiting (claiming the 1.43× back) or proceeds with informed consent. The commitment incentive is **risk-elastic** — a higher simulated default probability monotonically raises the reward — while staying capped at RM 15.00 and pool-bounded, so the pool is never overstated and money is never printed. The sub-additive, capped curve is deliberate: the marginal reward per additional risk percentage falls as risk grows, avoiding an incentive to gamble.

2.5 Zombie Radar — Recurrence & Telemetry Classification

Implemented in `lib/zombie-radar.ts`.

Detection (`detectRecurring`): group transactions by `normalizeMerchant(name)` (uppercase, alphanumeric only). A group is *recurring* when all of the following hold:

- `occurrences ≥ minOccurrences = 3`;
- every amount is within `±5%` of the group mean (`amountTolerancePct = 0.05`);
- consecutive intervals are each within `30 ± 3` days (`intervalDays = 30`, `intervalToleranceDays = 3`).

Classification (`analyzeZombies`), using `usageSignals.lastUsedDaysAgo` and the sandbox clock:

Condition	Tag
Recurring but no usage telemetry available	`SUSPECTED_ZOMBIE`
Recurring ≥ 3 months **and** last used > 30 days ago	`ZOMBIE`
Otherwise active	`ACTIVE`
`ACTIVE` but renewing within 3 days	`ENDANGERED`

A one-click answer ("have you used this in the past 30 days?") upgrades `SUSPECTED_ZOMBIE → ZOMBIE` on "no", or `SUSPECTED_ZOMBIE → ACTIVE` on "yes" (`applyUsageAnswer`). **Student arbitrage**: when the user pays full price and the registry exposes a student price, the subscription is added to `studentDowngradeCandidates` (e.g. Spotify RM 15.90 → RM 8.50). Merchant→ `subscriptionId` resolution is a hardcoded token table — no LLM, no network.

Clearing telemetry (Phase 8). Every `RecurringDetection` now exposes the settlement tolerances that make real card-rail recurrence detectable rather than guessed:

Field	Value	Meaning
`fxDriftTolerancePct`	`5`	Cross-border FX movement + card-scheme markup drift absorbed by the ±5% amount matcher
`settlementLagWindowDays`	`3`	28–31 day calendar months plus T+1/T+2 weekend clearing, inside the ±3-day interval matcher
`iso20022MandateCategory`	`pain.009_RECURRING_AUTODEBIT`	The ISO 20022 mandate message a production rail would carry for a recurring direct debit

These fields make the difference between "they charge me monthly" and a payment-clearing explanation a bank's operations team would recognise; the student-downgrade drawer surfaces the same telemetry when a downgrade is prepared.

---

3. Security & Compliance Architecture

3.1 Permission Tiers and the Master Autonomy Switch

Tier	Meaning	Enforcement
T0	Read-only analysis (What-If, budget insights, zombie analysis, roast)	Always allowed, still audit-logged
T1	Reversible, no money state change (zombie tagging)	Master ON **and** T1 enabled
T2	Preparation only (cancellation draft)	Master ON **and** T2 enabled
T3	Money state change (Quarantine / Escrow / Sweep)	Master ON **and** a valid per-action `USER_CLICK_APPROVE_PROPOSAL_*` reference at decision time

`assertTier(toolName, state, { authorizationRef })` in `lib/permissions.ts` is the single enforcement point. It runs inside every API route before a tool executes, and again inside the decision route with the user's click reference. The Master AI Autonomy Switch **defaults to OFF**, so the out-of-the-box state is T0 advisory only. The T3 gate requires *both* the Master switch ON *and* the click reference — creating a proposal is preparation, never authorization. The client-submitted autonomy payload is validated with `TierStateSchema` before `assertTier`; a malformed payload receives HTTP 400 instead of crashing or silently enabling a tier. T3 gate refusals are written to the audit chain with status **BLOCKED** and a reason-coded `actionType` (e.g. `T3_GATE_REJECTED_MASTER_AUTONOMY_OFF`).

3.2 Dual Limit Architecture

Two independent limits protect the user:

- **`UserSpendingLimit`** — the *user's own* declared weekly/monthly pay limits. Breaching either raises the risk level to at least HIGH and can trigger an intervention.

- `AiMandate` — the *user's* ceiling on what the agent may even *propose*: `maxLockAmountRM` and `maxLockDurationDays`. Lock categories are separately restricted to WANTS: NEEDS can never be escrowed or quarantined. The mandate is validated at proposal creation, re-validated at decision time, and can never be raised by the LLM.

3.3 Three-Stage Privacy & Hallucination Guardrails

1. **Pre-LLM PII redaction** (`redactPII` / `redactMessages`). Ordered regex rules mask Malaysian IC numbers (`YYMMDD-PB-####`), international and local phone numbers, 10–16 digit bank accounts and counterparty names after `TO` / `FROM`. Redaction happens in the browser before any text leaves the device and again on the server.
2. **Per-user session isolation** (`lib/session.ts`). The `userId` is derived only from a signed HMAC cookie (`ctrlpay_session`), never from the request body or the LLM. Every server read/write and every audit chain is keyed by this `userId`.
3. **Post-LLM number verifier** (`verifyNumbers`). Every `RM n` figure in the model's reply must appear (within ±RM 0.01) in the deterministic engine output for that turn or in the user's ledger. On failure the verifier records `NUMBER_VERIFIER_BLOCKED` in the audit log and the UI raises an observable verification notice; on success it records `NUMBER_VERIFIER_PASSED`.

3.4 Hardcoded Registry Whitelist and Zero-Write Cancellation

- **Whitelist.** All subscriptions live in `SUBSCRIPTION_REGISTRY` with team-maintained prices and **HTTPS-only** deep links. The LLM may reference a subscription only by its `subscriptionId` enum; it can never supply a URL, so prompt-injected links cannot reach the user.
- **Zero OAuth write scopes.** CtrlPay holds no autonomous credentials. Cancellations are produced as a client-side `mailto:` draft (`buildMailto`) that the user reviews and sends themselves — there is no Gmail send scope and no server-side email write path.
- **Tamper-evident audit.** `auditLog` chains entries as `entryHash = SHA-256(prevHash + canonicalJson(entry))`, so altering any earlier entry breaks every later hash and `verifyChain(userId)` returns `false`. Server-recorded AGENT entries are merged into the browser chain via `mergeExternal`, which recomputes hashes locally and is idempotent by `logId`, so the Audit Drawer always shows one unified, verified chain.

3.5 Serverless Proposal Integrity

On serverless platforms `POST /api/proposals` and `POST /api/proposals/[id]/decision` may run on different instances. The decision route therefore accepts the client-echoed `SignedProposal` and **always** recomputes `HMAC-SHA256(serverSecret, canonicalJson(terms) + proposalId + userId)` to verify the `integrityHash` before any state change. The proposal must be pending and unexpired, and the escrow floor / AI mandate are re-checked server-side. A single-byte change to any term (for example `premiumReward + 0.01`) is rejected with `INTEGRITY_HASH_MISMATCH`.

The API boundary also enforces the NEEDS invariant independently of the LLM: a direct `POST /api/proposals` carrying a NEEDS category for a Quarantine is rejected with `NEEDS_NOT_QUARANTINABLE` (HTTP 400), and unknown escrow categories are rejected as `INVALID_INPUT` rather than surfacing an uncaught schema error. Proposal **status** (pending/settled) is held in memory in this single-instance sandbox; production replaces it with a durable, tenant-scoped proposal table so replay protection survives cold starts.

---

4. Determinism, Testing & Reproducibility

- **Deterministic math.** All simulations use `mulberry32(seed)`; the default seed is `20260312`. Given the same inputs, the CFaR results, risk level and allowance are byte-for-byte reproducible. Phase 8's `executionTimeMs` is memoized with its result, and the stratified bootstrap uses a separate seeded PRNG, so not a single pre-existing engine output moved.
- **Test suite.** Vitest covers the engine, PRNG, signing/verification, SHA-256 (against `node:crypto`), canonicalisation, audit chaining, permissions, privacy guard, zombie radar, proposals, stateless decision, store/clock behaviour, tool schemas, bookkeeping and compliance. `tests/red-team.test.ts` exercises the real API routes for T-01 (Master OFF → 403 + BLOCKED audit), malformed autonomy → 400, direct NEEDS-Quarantine → 400 and unknown escrow categories → 400; `docs/RED_TEAM_TEST_MATRIX.md` maps all ten red-team checks to executable tests. `tests/fintech-hardening.test.ts` (Phase 8) additionally locks: ES95 ≥ CFaR95 coherence across scenarios, the pinned sandbox regression values, intra-month barrier detection of a day-2 bill that clears before the day-25 allowance lands (and the inverse case), weekend stratification, Laibson β/1.43× telemetry with a capped risk-elastic reward, the Stone-Geary floor fields, the sinking-fund maturity gate and the pre-auth latency budget.
- **Compliance guard.** A test asserts the AI SDK v4 client identifiers (`addToolResult`, `maxSteps`), explicit `parameters` function declarations, the *absence* of v5 identifiers (`inputSchema`, `addToolOutput`, `toUIMessageStreamResponse`, `stopWhen`), the Groq GPT-OSS 120B default, and that free-tier safety never reads the `plan` flag.
- **Evidence.** The judge walkthrough is reproducible via the sandbox execution-evidence generator (see `docs/SANDBOX_EXECUTION_EVIDENCE.md` and `docs/EXECUTION_EVIDENCE_SANDBOX_LOGS.json`). The in-app **Audit Log** → **Export evidence** button also downloads the live hash chain as JSON.

---

5. v2.5 Feature Extensions

5.1 Multi-currency & Purchasing-Power-Parity (PPP) display layer

Invariant — single base ledger. The Zustand store, ledger, `SUBSCRIPTION_REGISTRY` and `quant-engine` always compute in **MYR (RM)**. Currency switching is a pure render-layer mapping in `lib/currency-ppp.ts`; it never mutates state, never re-signs a proposal and never changes what is sent to the LLM. This keeps the Post-LLM numeric-provenance guard and every HMAC-SHA256 `integrityHash` valid.

For a base amount v_{MYR} and region c:

$$v_{\text{display}}(c) = \operatorname{round}(v_{\text{MYR}} \times r_c,\ 2)$$

$$N_{\text{meals}} = \max\left(1, \operatorname{round}\left(\frac{v_{\text{MYR}}}{9.00}\right)\right)$$

The canteen meal index is anchored to the Malaysian campus baseline (RM 9.00/meal) while the caption localises the per-meal price using the regional registry:

Code	Region	Symbol	Rate vs MYR	Local canteen meal	Survival floor
MYR	Malaysia (ASEAN)	RM	1.00	RM 9.00	RM 15.00
CNY	Shenzhen / Mainland (GBA)	¥	1.65	¥ 15.00	¥ 25.00
HKD	Hong Kong (GBA)	HK$	1.80	HK$ 40.00	HK$ 65.00

Code	Region	Symbol	Rate vs MYR	Local canteen meal	Survival floor
SGD	Singapore (ASEAN)	S$	0.30	S$ 5.00	S$ 8.00
USD	Global	$	0.23	$ 10.00	$ 16.00

Every switch appends a T0 read-only `CURRENCY_VIEW_SWITCH` audit entry (`data_accessed: ['displayCurrency']`). The converted values are rendered in the Home balance, the Dynamic Daily Allowance, the Defense Vault hero and the payment-interceptor CFaR panel.

5.2 iOS Back-Tap quick logging & live Dynamic Budget Management

`components/quick-log-backtap-modal.tsx` accepts three equivalent triggers:

1. **iOS Shortcut / Back-Tap deep link** — open `/?trigger=backtap` (bind it to *Settings → Accessibility → Touch → Back Tap*).
2. **Sandbox button** — Home header and Judge Sandbox → Time Machine.
3. **Desktop shortcut** — double-tap the `B` key (ignored while typing in an input).

The modal previews the post-entry daily allowance before committing:

$$\text{DDA}*{\text{projected}} = \frac{\max(0,\ B_t - X*{\text{entry}} - \sum F_i - S_{\text{goal}})}{\max(1, D_{\text{rem}})}$$

Committing calls `quickLogTransaction` , which writes the **single MYR ledger** (expense or income), emits a `BOOKING` sandbox event and appends a T1 `USER_BACKTAP_QUICK_LOG` / `USER_BACKTAP_INCOME` audit entry with `authorization_ref = USER_AUTHORIZED_BACKTAP_TRANSFER` . The next render recomputes `dynamicDailyAllowance()` from the engine — no cached or LLM-derived budget is used.

5.3 Zero-knowledge passkey gate & T3 biometric signing

`components/biometric-passkey-gate.tsx` provides:

- a black-glass **Face ID / Touch ID gate** (locked status-bar icon), with a 0.8s simulated scan; and
- a **0.5s signing pulse** shown while a T3 contract is approved.

SIMULATION ONLY. No WebAuthn credential, biometric template or device key is created, stored or transmitted. The compliance statement is rendered in the UI: "*Zero-Knowledge Local Enclave Simulation (WebAuthn / FIDO2 standard) · Biometric templates never leave the user's device or enter CtrlPay servers.*"

The T3 gate now accepts two action-specific ref formats:

- `USER_CLICK_APPROVE_PROPOSAL_<id>` — plain per-action click (default).
- `FIDO2_PASSKEY_BIOMETRIC_AUTH_<id>` — the same action performed through the passkey gate.

`lib/permissions.ts` validates both against `CLICK_AUTH_RE` ; the server still enforces Master ON + T3, and the client audit entry records the structured ref so the audit drawer shows the biometric attestation path. Swapping the simulated scan for the real WebAuthn API requires no change to the permission or audit layers.

5.4 Assisted cancellation sheet (T2 draft + T3 two-stage settlement)

`components/subscription-cancel-email-sheet.tsx` upgrades the former student-downgrade drawer into a two-channel sheet:

- **Channel A — whitelisted deep link.** The only URL rendered comes from `SUBSCRIPTION_REGISTRY` by `subscriptionId`; the LLM never supplies a URL. Student plans additionally require a mock `.edu` OTP (any 6 digits in the sandbox) before the downgrade action is enabled.
- **Channel B — in-app email preview.** When the registry has a `mailtoTemplate`, the full `To / Subject / Body` draft is rendered inside the sheet (solving the "mailto is invisible on a judge's laptop" problem) with an "Open in mail app" link and an **"I've sent it — start PendingSweep"** button.

Both paths create a server-signed **SWEEP** proposal (`STUDENT_DOWNGRADE` or `CANCEL`) and apply it as a `PENDING_NEXT_BILLING_VERIFICATION` record. Nothing enters `defenseVault` until the sandbox clock crosses the next billing date and the charge verifiably disappears.

5.5 Execution evidence & export

The deterministic generator now runs an **8-step** workflow: default-OFF gate → Shopee RM 180 interception → 24h quarantine → +24h reward claim → student downgrade sweep → month-end two-stage settlement → **Back-Tap quick log with DDA re-computation** → SHA-256 chain verification. It writes `docs/EXECUTION_EVIDENCE_SANDBOX_LOGS.json` and `docs/SANDBOX_EXECUTION_EVIDENCE.md`. Judges can also download the live chain from the Audit Log panel.

5.6 Topic A four-verb coverage

Topic A requirement	CtrlPay implementation
Budgeting	Dynamic Daily Allowance, dual mandate limits, sinking fund, Needs-Protected Escrow
Bill analysis	Fixed-bill calendar, subscription registry, two-stage sweep verification, Zombie Radar
Spending insights	CFaR / ES95 bootstrap simulation, corridor chart, student-unit equivalents, month-end roast
Actionable recommendations	Signed T3 contracts (quarantine / escrow / sweep), assisted cancellation email, deep links, Back-Tap capture

---

Prices, thresholds and fixtures in this document are sandbox/demo values maintained by the team, not live market data.